

RESEARCH ARTICLE

Ensemble Balanced Nested Dichotomy Fuzzy Models for Software Requirement Risk Prediction

GANESH KUMAR¹, ABDULLAHI ABUBAKAR IMAM², SHUIB BASRI³, (Member, IEEE),
AHMAD SOBRI HASHIM³, ABDUL GHANI HAJI NAIM²,
LUIZ FERNANDO CAPRETZ⁴, (Senior Member, IEEE),
ABDULLATEEF OLUWAGBEMIGA BALOGUN³, (Member, IEEE),
AND HUSSAINI MAMMAN³

¹Department of Computing and Information Systems, Sunway University, Subang Jaya, Selangor 47500, Malaysia

²School of Digital Sciences, Universiti Brunei Darussalam, Bandar Seri Begawan BE1410, Brunei Darussalam

³Department of Computer and Information Sciences, Universiti Teknologi PETRONAS, Seri Iskandar 32610, Malaysia

⁴Department of Electrical and Computer Engineering, Western University, London, ON N6A 5B9, Canada

Corresponding author: Abdullateef Oluwagbemiga Balogun (abdullateef.ob@utp.edu.my)

This work was supported in part by Universiti Teknologi PETRONAS, Malaysia, under the Short Term Internal Research Funding (STIRF) under Project 015LA0-049; and in part by the Ministry of Higher Education, Malaysia Fundamental Research Grant Scheme under Grant 015MA0-170.

ABSTRACT Modern software systems are becoming more intricate, making identification of risks in the software requirement phase—a fundamental aspect of the software development life cycle (SDLC)—complex. Inadequate risk assessment may result in the malfunction of a software system, either in the development or production phase. Therefore, risk prediction plays a crucial role in software requirements, serving as the first step in any software project. Hence, developing adaptive predictive models that can offer consistent and explainable insights for handling risk prediction is imperative. This study proposes novel ensemble class balanced nested dichotomy (EBND) fuzzy induction models for risk prediction in software requirement. Specifically, the proposed EBND models employ a hierarchical structure consisting of binary trees featuring distinct nested dichotomies that are generated randomly for each tree. Thereafter, we use an ensemble principle to refine rules generated from the resulting binary tree. The predictive efficacy of the suggested EBND models is further extended by introducing a data sampling method into their prediction process. The inclusion of the data sampling method acts to mitigate the underlying disparity in the class labels that may affect its prediction processes. The efficacy of the EBND models is then evaluated and compared to current solutions using the open-source software risk dataset. The observed findings revealed that the EBND models demonstrated superior predictive capabilities when compared to the conventional models and state-of-the-art methodologies. Specifically, the EBND models achieved an average accuracy threshold value of 98%, as well as high values for the f-measure metric.

INDEX TERMS Software requirement, software risk prediction, machine learning, ensemble learning.

I. INTRODUCTION

By incorporating Artificial intelligence (AI) into software engineering (SE) processes, organisations can anticipate enhanced product quality, accelerated time-to-market, and a

The associate editor coordinating the review of this manuscript and approving it for publication was Ali Kashif Bashir¹.

competitive advantage in the swiftly changing technology environment [1]. Software development teams are then able to access unprecedented levels of productivity and innovation by understanding and utilizing AI across the Software Development Life Cycle (SDLC) [2]. This application cuts across all phases of the SDLC, leading to enhanced decision-making processes, and minimizing the likelihood of errors that may

be detrimental to the quality of the software system. For instance, IBM Watson, a natural language processing (NLP) and cognitive computing AI tool, can analyze user feedback and product reviews to identify implicit requirements that may not have been explicitly stated by stakeholders during the requirement gathering process. This helps to create a more comprehensive set of requirements [3]. In terms of software design, AI tools such as DeepCode can suggest optimal software architectures based on best practices and the specific needs of a project, ensuring that the design is robust and scalable [4]. GitHub Copilot can autocomplete code, generate entire functions, and suggest best practices, significantly speeding up the coding process [5]. DeepCode and Codacy analyze code for potential bugs, suggesting improvements and ensuring code quality [6]. Organizations such as Facebook use Sapienz, another AI tool, to analyze code changes and predict where bugs are likely to occur, allowing developers to address issues before they affect the software [7]. Likewise, the EvoSuite AI tool generates unit tests that maximize code coverage, identifying edge cases that manual testing might miss [8]. Test.ai uses machine learning to analyze past test results and optimize the order in which test cases are executed, focusing on the most critical areas first [9]. These are proven use cases of the successful integration of AI into the critical phases of the SDLC to ensure a high-quality product.

However, software system development often involves a high level of ambiguity due to unpredictable circumstances that might arise at any phase of the SDLC. Due to the growing intricacy and interconnectivity of contemporary software systems, the increasing rates of failure in software projects have become a significant concern [1], [2]. The term “software risks” refers to the unforeseeable uncertainties that arise due to the diverse risk factors inherent in the various stages of the SDLC [10], [11]. Significantly, these uncertainties have led to substantial financial losses and inefficiencies for software enterprises. As a result, it is crucial to swiftly and systematically deal with these uncertainties, since neglecting to do so may ultimately lead to the failure of the software system [12].

According to research conducted by the Standish Group, thirty of every one hundred software initiatives fail before completion, while just sixteen of every one hundred are completed with no issues. Global organizations face a significant challenge, with just nine out of one hundred software projects effectively meeting both the stipulated timetable and budget constraints. It is worth noting that even completed software projects often fail to fully address the initially stipulated software requirements. Only one-third of US technology businesses are reported to complete the implementation of technical specifications in software systems. One viable strategy for addressing software risk issues is the implementation of risk assessment procedures [13], [14].

Risk assessment is a systemic procedure that analyzes any potential dangers linked to a scheduled task in the SDLC [15]. The evaluation might be conducted using a proactive or

reactive method. Experts view the latter technique (reactive) as an immature risk assessment strategy due to its tendency to prolong the project timelines, require additional resources, and compromise the system’s standards and stability. However, the former assessment strategy (proactive) uses explicit means such as statistical or machine learning (ML) methods to minimize the likelihood of system malfunction [16]. Prompt risk detection is beneficial across all phases of the SDLC, especially during the system specification-gathering period, since it impacts the whole SDLC [14], [17].

The utilization of independent assessment or judgement by experts or professionals is a commonly employed technique in contemporary project risk management [18], [19]. The system’s effectiveness is contingent on an individual’s expertise, making it inherently reliant on human involvement, which can result in responses or insights that lack clarity or precision [20]. Additionally, the system cannot be replicated, as the transfer of expertise across different units involved in a software project is not a straightforward process [21]. It is essential to create adaptive ML techniques that are capable of offering more unbiased, reproducible and explainable decision-making assistance for risk management [10], [14].

The SDLC has deployed and suggested numerous methodologies for anticipating software risk. For instance, Hu et al. [22] explored the use of support vector machines (SVM) and artificial neural networks (ANN) to predict uncertainties at the requirement stage of the SDLC. The suggested approaches were applied to the set of data collected from survey responses. In a similar vein, there have been proposals for employing ensemble methods that rely on Bayesian network (BN) models for prediction of software risks [13]. Influence diagrams (ID) and BN have also been employed in the domain of software project risk prediction. Nevertheless, it is important to acknowledge that both the ID and BN models possess inherent drawbacks. Specifically, these models (ID and BN) necessitate prior information in their respective working operations and encounter challenges when it comes to scalability [23], [24]. In contrast, models such as ANN are frequently used and efficient for predicting risks. In their study, Lee et al. [24] proposed an ANN model to predict programme sub-module outcomes. They observed and reported that the ANN model exhibited superior performance compared to traditional techniques like decision trees (DT) and multiple linear regression (MLR) in tackling complex challenges related to risk prediction in software projects [24], [25]. However, there is a continuous and critical need for better and scalable solutions for optimal risk prediction [10], [14]. In addition, the successful implementation and efficiency of utilising an effective ML model depend on the choice of model and the training dataset used by such a model [26], [27]. Several ML models and methods have been utilized in the realm of software risk prediction, exhibiting similar levels of predictive performance and relative success stories. For example, data quality issues, such as inconsistencies in category labels or redundant features, may

cause the phenomenon and hinder the effectiveness of ML technique [28].

Therefore, this study presents ensemble balanced nested dichotomy fuzzy induction (EBND) methods for predicting the risks and uncertainties associated with software requirements. Specifically, this research uses the Fuzzy Unordered Rule Induction Algorithm (FURIA), Ripple Down Rule Learner (RIDOR) fuzzy models, and their respective enhanced EBND variants for risk prediction at the requirement gathering stage of an SLDC. FURIA is an algorithmic model that uses fuzzy rules and a proficient rule expansion technique to facilitate its classification procedures [29]. These fuzzy rule methods are known to exhibit a greater level of generality as compared to traditional if-then rule methods, as they set flexible limits for categorization processes. In contrast, traditional if-then rule methods adhere to rigid decision boundaries that display abrupt transitions between distinct classes, leading to the creation of models that may be considered questionable and lacking in intuitive interpretation. Hence, the comprehensibility of FURIA presents a significant benefit compared to the aforementioned alternative approaches [30]. This attribute has played a crucial role in the decision to adopt FURIA in the context of this research.

Additionally, RIDOR is an incremental, knowledge representation ML model whose primary goal is to learn and extend a set of if-then rules generated from a given dataset. The hierarchical structure organizes these rules, associating each rule with a specific class or decision. The hierarchical nature of the RIDOR model allows it to represent complex and overlapping decision boundaries efficiently. For example, it can revise and refine its rules when presented with new instances that contradict the existing rules [31]. This adaptability allows the RIDOR model to improve its performance over time, as well as handle large datasets and adapt to new patterns efficiently, making it suitable for real-world applications with dynamic data environments [32]. Furthermore, the proposed EBND methods utilize a binary tree structure, linking each node to a specific class group. A binary classifier divides the classes into two distinct subsets within the internal nodes of the EBND tree, assigning these subsets to their respective child nodes. The EBND system thus integrates multiple nested dichotomies, randomly generating each tree [33].

The prediction process incorporates a data sampling technique to improve prediction performance and address any inherent data quality issues in the dataset, thereby mitigating the latent class imbalance problem. We specifically deployed the Synthetic Minority Oversampling Technique (SMOTE), a method frequently employed to address class imbalance in data augmentation, [34]. SMOTE functions by generating artificial instances for the underrepresented class label to balance the class label proportion. We generated the newly created instances by interpolating original instances of the underrepresented class labels [35]. Hence, SMOTE efficiently augments the number of instances in the minority class to achieve a more balanced distribution compared to

the majority class. By doing this, it assists the model in learning from a wider range of instances of the minority class, which may result in enhanced generalization and increased performance on imbalanced datasets [36].

In summary, this study has made significant scientific contributions, which can be summarized as follows:

- The development and detailed assessment of FURIA and RIDOR models for software requirement risk prediction.
- The formulation, deployment, and assessment of novel Ensemble class Balanced Nested Dichotomy (EBND) for software requirement risk prediction.
- The implementation of SMOTE for the alleviation of class imbalance and enhancement of the effectiveness of EBND variants of FURIA and RIDOR for effective software requirements risk prediction.
- The empirical comparison and validation of the suggested Ensemble Balanced Nested Dichotomy Fuzzy Induction Models with standard and current state-of-the-art methods for software requirement risk prediction.

The remaining portions of this paper are arranged as follows: Section II of the document encompasses the presentation and discussion of studies that are closely connected and pertinent to the topic at hand. Section III contains a comprehensive account of the experimental procedure and methods employed. Section IV provides a comprehensive overview of the experimental outcomes and the corresponding research observations. The final section of this study serves as the conclusion and provides recommendations for further study.

II. RELATED WORK

Risk analysis methods have been effectively used to forecast the outcomes of software development projects [13], [22], [23], [24], [25], [37]. Khan and Faisal [37] tried to identify which risks are significant and how frequently they occur by exploring and identifying the circumstances in which the risks might result in software failure during the design process. They also proposed a ML model, named the Risk Prediction Trees (RPTs), that can forecast risks throughout the design phase [37]. The suggested RPT is based on the BN approach. However, models based on BN, BBN, and ID have a problem in that they rely on the user having prior knowledge of the connections involving risky elements and the impact of variables on the result. The conditional probability table and the learned links between them are highly idiosyncratic products of the practitioner's experience. As a result, the expansion of nodes in BBN is constrained by the requirement for additional study to construct a new conditional probability table.

A study by Hu et al. [22] discovered the high failure rates and inherent risks of software project development by devising a smart model that could foresee and control the hidden dangers of software development initiatives in order to lessen them. According to Kawamura et al. [38], more than 70% of software projects fail. To address this problem, they conducted an online survey of 332 software projects and used the results to train an ANN classifier for use in predicting

software risk. They concluded that software risk prediction may help businesses prioritize software initiatives based on risk. Christiansen et al. [39] also used the MLR to assess software development risk based on expert survey data. Using statistical integration, they demonstrated the predictability and control of risk factors via risk mitigation at every phase of the SDLC. By combining risk factor representation and MLR, they were able to predict the likelihood of success or failure for software endeavors. Their research highlighted the need to identify and control the risks involved in software development processes to ensure timely completion of software projects. Nonetheless, as most of this research relies on responses to surveys, the results should be interpreted cautiously.

In a related study, Khan et al. [40] used tree-based ML techniques for predicting risks in software processes. They deployed ten alternative tree-based techniques with varying computational characteristics to predict the risk of software requirements. They conducted research on the premise that tree-based models are more adept at handling continuous and categorical input and output variables compared to other models. In comparable research, Qureshi et al. [41] investigated the performance of prominent ML models such as instance-based, function-based, and ensemble-based models for software risk prediction. Their findings demonstrated the usefulness of the implemented ML models. However, tree-based models are prone to generating overfit models that may exhibit bias towards features with high representation. Additionally, the issue of parameter tuning in instance and function-based models is a limitation of these ML techniques. Moreover, most ML models do not perform well in the presence of data quality issues such as class imbalance and high dimensionality problems.

Mahmud et al. [42] presented a detailed literature review to enhance understanding of software risk prediction, showcasing data analysis and approaches deployed from 2007 to 2022. Findings from this review further validate the critical nature of the risk, not only in the requirement phase but in every other phase of the SDLC. Shaikat et al. [43] highlighted the requirement elicitation step of the SDLC as the most crucial and challenging process for resolving data challenges in software risk prediction. Further investigations revealed that no unified dataset culled from actual software projects captured the nuances of software requirements and risks. A dataset including needs obtained from the Software Requirement Specification (SRS) of specific software systems and their risk characteristics was proposed and integrated into the requirement-gathering phase of the SDLC [43]. Alharbi et al. [10] investigated the uncertainties that may emerge in handling concurrent software initiatives. Their experiment showed very high degrees of precision. The risk prediction accuracy of logistic regression (LR) was 93%, whereas that of REPTree was 98% in assessing the relative dangers of several ongoing projects. In addition, Xu et al. [44] combined the genetic algorithm (GA) and the DT technique, which can choose the best set or collection of metrics to use

for predicting software risk on their own. The experimental results showed that the strategy worked, with significant improvements in prediction performance demonstrated.

For risk analysis in software projects, Hu et al. [18] introduced a BN-based framework with causality constraints (BNCC). The recommended model not only beats existing ML approaches like LR, DT, NB, and traditional BN, but also finds causalities based on expert knowledge. In a comparable study, Akumba et al. [45] employed a NB classifier to forecast risk in the requirement gathering stage of an SDLC. To classify risks as catastrophic, high, moderate, low, or unimportant, the NB model was developed using criteria such as magnitude, impact, probability, priority, and risk dimension. According to their findings, probabilities and priorities are the most significant indicators of future risk. In other words, knowing that the danger is significant and noting its importance in advance are both helpful. Naseem et al. [14] conducted a comprehensive empirical investigation on ML approaches for software risk prediction. The results of the experiments showed that ML methods are applicable for predicting software vulnerabilities.

Otoom [46] also developed an ensemble technique based on Adaboost and DT for predicting software risks. The suggested method's prediction performance was contrasted with standard models and found to be better. The closest to this research is the work of Mamman et al. [47]. The authors suggested fuzzy induction models using layered subset selection dichotomies for software requirement risk prediction. The proposed fuzzy induction models use efficient rule-stretching techniques for the prediction function. The observed research findings showed that fuzzy induction models are better and more efficient. However, most of the existing research on ML-based software risk prediction has ignored the class imbalance problem that exists in the software risk datasets. However, Naseem et al. [14] noted that the performance of ML may be challenged when the software risk dataset has imbalanced classes (labels). Existing research has also shown that models constructed using imbalanced datasets are more likely to overfit or give unreliable results [34], [36]. Hence, this study proposes the use of advanced fuzzy induction models to predict software risks. Ensemble techniques are known to have superior predictive capacity as compared with single or individual ML models in most cases. As a result, this study presents and employs improved fuzzy induction models such as FURIA and RIDOR, as well as their respective EBND variations, along with the SMOTE data sampling method to predict the likelihood of risk in the software requirement process. Table 1 provides a comprehensive assessment of the literature, facilitating a deeper understanding of the primary outcomes and limitations.

III. METHODOLOGY

This section presents a comprehensive discussion of the research methods used for this research. It also provides a detailed explanation of the developed FURIA, RIDOR, and their EBND versions, along with the experimental methodology.

TABLE 1. Detailed analysis of existing literature.

References	Models	Data Quality Issues	Findings	Limitations
Boehm [49]	Risk Management Technique	No	The recognized fundamental components are risk identification, assessment, and mitigation.	Subject to and can be influenced by expert knowledge base and opinion.
Khan and Faisal [37]	BNM	No	Categorization and assessment of risk into a RPT.	BNM rely heavily on the prior knowledge of the connections involving data elements and the impact of variables on the result
Hu, et al. [22]	SVM and ANN	No	Effective SVM/ANN ML models were proposed	Both SVM and ANN have perceptibility issues, which hinder the comprehension of the prediction results and makes it difficult to replicate the underlying issues.
Kawamura, et al. [38]	NB	No	An intelligent Bayesian classifier was developed	The experimentation was limited in scope and the result lacks generalizability.
Christiansen, et al. [39]	MLR	No	Utilized MLR to discover and mitigate intrinsic risk indicators.	MLR is sensitive to the nature of the dataset and does not work well with categorical data
Hu, et al. [13]	Cost-Sensitive Model and Hybrid Ensemble	No	The proposed system, called COSENS, combines cost-sensitive and ensemble-based modelling techniques to estimate software project risk.	Assessing the expenses or fines for misclassification can be biased and might change depending on the viewpoints of various stakeholders.
Bai [23]	Enhanced MBN	No	MBN have been utilized for predicting software dependability.	MBN are known to have scalability and interpretability problems
Lee, et al. [24]	BBN	No	An extensive project risk management approach utilising a BBN.	BBN are known to have scalability and interpretability problems
Vera and Del Carpio [50]	LR, MLP, DT, RF, and SVM	No	Comparison of multiple ML methods for software development risk prediction.	The experimentation was limited in scope and the result lacks generalizability. In addition, the implemented models are affected with change in parameter settings/tuning.
Khoshgoftaar, et al. [25]	Neural Network	No	Enhanced measurement for early risk assessment of latent defects	The experimentation was limited in scope to a case study. Hence, the result lacks generalizability.
Khan, et al. [40]	CDT, CS-F, DS, F-PA, HT, J48, LMT, RF, RT, REP-T	No	Comparison of multiple Tree-based ML methods for software development risk prediction.	The experimentation was limited in scope as only tree-based ML models were considered. Also, the results cannot be generalized since it was not compared with existing studies.
Qureshi, et al. [41]	LDA, SVM, SGD, RF, GNB, KNN, DT, LR, NB	No	Comparison of multiple ML methods for software development risk prediction.	The experimentation was limited in scope and the result lacks generalizability. Further, the implemented models are affected with change in parameter settings/tuning.
Shaukat, et al. [43]	kNN	No	Validation of software risk dataset	Only kNN was implemented for the validation of the proposed dataset.
Alharbi, et al. [10]	LR and RepTree	No	Analysis of risk assessment in concurrent software projects	LR is reported to act poorly with categorical data and RepTree has the tendency to overfit.
Xu, et al. [44]	Hybrid (GA+DT)	Feature Selection	Reduced features with improved predictive performance	GA is known to have convergence problem. Moreover, DTs are unstable in performance when features are reduced.
Hu, et al. [18]	BNCC	No	BNCC was superior to other experimented models	BN are known to have scalability and interpretability problems
Akumba, et al. [45]	NB	No	Applicability of NB in the prediction of risk in the requirement specification phase	The experimentation was limited in scope and the result lacks generalizability.
Naseem, et al. [14]	DTable, AIDE, DT, RF, NB, KNN, CHIRP, DTNB, CDT, CS-F	No	Comparison of multiple ML methods for software development risk prediction.	The experimentation was limited in scope and the result lacks generalizability. In addition, the implemented models are affected with change in parameter settings/tuning.
Otoom [46]	Boosting+DT	No	Homogeneous ensemble method (boosting) for software risk prediction	Data quality issues such as class imbalance was not considered.
Mamman, et al. [47]	FURIA (FCS, BSS, RPS)	No	Enhanced fuzzy induction models based on nested dichotomies for software requirement risk prediction	Data quality issues such as class imbalance were not considered.
Binsawad and Khan [51]	ForExPlusPlus (FEPP)	No	Compact rule extraction and multi-class integration	Data quality issues such as class imbalance were not considered.

A. FUZZY UNORDERED RULE INDUCTION ALGORITHM (FURIA)

FURIA is an improved variant of the RIPPER and IREP algorithms and was developed as a solution to the underlying issues that existed in the algorithms around the process of rule formation and the use of a single class for the main prediction. It is well established that difficulties of this kind result in a systematic bias towards the default class designation [30], [48].

On the other hand, FURIA has a propensity to differentiate between class labels without taking into consideration the default rule or the sequence in which the class labels appear. In most cases, FURIA will implement an extra rule stretching function, which gives it the ability to broaden the scope of the created rules such that they apply to all data instances [52]. FURIA employs pruning in the process of creating the replacement and revision rule to fulfil its pruning requirements. In this scenario, the initial pruning method is used; however, an exception is made if the pruning strategy attempts to get rid of all antecedents associated with a rule, which results in the generation of a base rule [51]. That is, the pruning operation ceases, and the base rule can be utilized for the comparison of the Minimum Description Length (MDL) during the model's improvement process. These pruning procedures are sufficient to prevent the resulting model from overfitting. To generate the fuzzified rules, FURIA seeks the optimal fuzzy extension of the base rules. A fuzzy extension is implemented by applying a rule that has a similar structure but includes a fuzzy interval. For instance, with intervals A_i on the base rules as the centre $[\partial_x^{a,B}, \partial_x^{a,C}]$ of the explored intervals A_i^F , determining the best limits for the created supports is the main concern. To fuzzify an individual antecedent ($H_j \in A_i$), it is vital to examine only the essential instances T_i^j .

$$T_i^j = \{y = (y_1 \dots y_k) \in T_k | A_i^F(y_j) > 0 \text{ for all } j \neq i \subseteq T_i\} \quad (1)$$

T_i^j is grouped based on positive (T_{i+}^j) instances and negative (T_{i-}^j) instances.

To ascertain the level of fuzzification, the rule purity (p) is used:

$$p = \frac{r_i}{r_i + m_i} \quad (2)$$

where

$$r_i = \sum_{y \in T_{i+}^j} \alpha_{H_j}(y) \quad (3)$$

$$m_i = \sum_{y \in T_{i-}^j} \alpha_{H_j}(y) \quad (4)$$

The process of fuzzy inference is then applied to the antecedent that has the highest purity. The process will be repeated until all antecedents are "fuzzied." It is vital to note that a straightforward fuzzification is constantly observed by shifting the support bound to the occurrence that comes immediately after the core bound. This method of data fuzzification helps identify new cases, although it does not compromise the integrity of the training data.

FURIA utilizes minimal refinements of rules to identify suitable substitutes for every data item. It is possible to derive a refinement of a rule, often known as "stretching" the rule, by removing certain numbers of the rule's predecessors. Refinement is considered basic if it only eliminates the necessary predecessors to constitute the question data item. Consequently, the minimum refinement of a rule may be determined by eliminating all the predecessors for which the inquiry does not provide a satisfactory response. Further details on FURIA may be seen in [29], [30], and [53]. Algorithms 1 and 2 present the pseudocodes for FURIA and its fuzzification process, respectively.

Algorithm 1 FURIA Pseudocode [48]

Input:

- Training dataset T with instances (x, y) where x is a feature vector and y is the class label
- Set of classes C
- Maximum number of rules R_{max}
- Number of optimization iterations I_{max}

Output:

- Fuzzy rule set FRS

1. Initialize FRS as an empty list.
 2. For each class c in C :
 - a. Extract subset T_c of T where class label is c .
 - b. While T_c is not empty, and the number of rules is less than R_{max} :
 - i. Generate a fuzzy rule R for class c :
 - Initialize R as an empty rule.
 - While the rule R does not cover any instances in T_c :
 1. Select a feature f from the feature set.
 2. Define a fuzzy set F for feature f .
 3. Add the condition (f is F) to rule R .
 4. Evaluate rule R on T_c .
 5. If R does not cover any instances, modify F to increase coverage.
 - ii. Optimize rule R :
 - Perform I_{max} iterations of optimization:
 1. Adjust fuzzy sets in R to maximize coverage and accuracy.
 2. Prune conditions in R to avoid overfitting.
 - iii. Add rule R to FRS.
 - iv. Remove instances from T_c covered by rule R .
 3. Return FRS as the final fuzzy rule set.
-

B. RIPPLE DOWN RULE LEARNER (RIDOR)

The RIDOR algorithm, created by Gaines and Compton [54], aims to set a base rule and then identify any exceptions to this rule while minimising the weighted error rate. It generates the optimal exceptions for every potential exception and continues iterating until it achieves the final state. Consequently, it extends exceptions in a manner akin to a tree [55]. The

Algorithm 2 Fuzzification of FURIA [29]

```

Let Y represent the numerical predecessors of x
While  $Y \neq \emptyset$  do
     $y_{max} = \text{NULL}$ 
    //  $y_{max}$  denotes the antecedents with the highest purity.
     $purity_{max} = 0$ 
    //  $purity_{max}$  denotes the highest purity value.
    for  $i = 1$  to Size (Y) do
        Compute the fuzzification of  $Y[i]$  in terms of purity
        Let  $purity_{Y[i]}$  be the fuzzification with the best purity
        if  $purity_{Y[i]} > purity_{max}$  then
             $purity_{max} = purity_{Y[i]}$ 
             $y_{max} = Y[i]$ 
        end if
    end for
     $Y = Y_{y_{max}}$ 
end while

```

exceptions constitute a collection of rules that may conduct categorization in addition to the base rule. Incorrect occurrences of base rules may be anticipated with the use of a set of rules known as “the exceptions”, which are generated via the usage of IREP [54]. The initial intent of RIDOR was to make rule-based systems easier to understand and keep up to date. It can be challenging to acquire information and incremental rule learning to add new rules without affecting the consistency of the rule foundation, perhaps leading to poor performance of current rules in the new induction process. Ripple-down rules, which are derived from a covering approach for rule set creation, differ from traditional induction rules by allowing exceptions to the rule without affecting the rule outside of its immediate context.

Ripple-down rules are extended by adding exception or “else” branches to the existing rules, as when decision lists might result in “if-then-else” rules. A new rule’s “an except” branch is created if it executes but leads to the wrong result. If no preexisting rule triggers the new rule, the “an else” branch is generated. Additional information about RIDOR may be found in [54] and [56]. Algorithm 3 presents the pseudocode for RIDOR.

C. ENSEMBLE BALANCED NESTED DICHOTOMY (EBND)

The Ensemble Nested Dichotomy (END) method converts a multiclass problem into binary classification tasks using a binary tree structure, where the root node encompasses all classes in the multiclass dataset. Next, a model is developed to differentiate between the two sets of classes (termed super classes) that have been created from the original set. Each terminal node in the hierarchy has just one class as the process is iteratively applied, partitioning higher-level classes until a single class is formed from the initial dataset. The tree is traversed using a binary structure to determine which suitable branch to traverse at each level to categorize a new item. When an object reaches a leaf, it is categorized under that specific leaf.

Algorithm 3 RIDOR Pseudocode [57]

```

Input:
- Training dataset T with instances (x, y) where x is a feature vector and y is the class label
Output:
- Ripple Down Rule set RDRS
1. Initialize RDRS as an empty list.
2. Determine the default rule D:
    a. Identify the most frequent class in T, denoted as default_class.
    b. Set D as the rule “IF True THEN default_class”.
3. Add D to RDRS.
4. Generate exception rules for the default rule D:
    a. Initialize exceptions for D as an empty list.
    b. Identify instances in T misclassified by D, denoted as misclassified_instances.
    c. While misclassified_instances is not empty:
        i. Generate an exception rule E for the most frequent class in misclassified_instances:
            - Initialize E as an empty rule.
            - While E does not cover any misclassified_instances:
                1. Select a feature f from the feature set.
                2. Define a condition (f = value) where value is from the misclassified_instances.
                3. Add the condition to rule E.
                4. Evaluate rule E on misclassified_instances.
                5. If E does not cover any instances, modify the condition to increase coverage.
        ii. Optimize rule E:
            - Prune conditions in E to avoid overfitting and to simplify the rule.
        iii. Add rule E to the exceptions of D.
        iv. Remove instances from misclassified_instances covered by rule E.
    d. Add the exceptions of D to RDRS.
5. For each exception rule in RDRS, repeat step 4 to generate further exception rules.
6. Return RDRS as the final Ripple Down Rule set.

```

ENDs have recently been shown to be a robust replacement for paired classification and conventional error-correcting output codes. The idea behind the END method is that more divergent groups may be effectively divided into smaller, more homogenous groups within a series of nested dichotomies partitioned at the deeper levels of a nested dichotomy. Finding the centroid of each group allows us to determine how far apart they are. Using these measures of separation, we may divide the classes into two clusters, with a_1 and a_1 as the nodes.

For a problem with x class labels, it stands to reason that the tree has x leaf nodes and x-1 interior nodes. There is a binary classifier inside each internal node. Employing an ensemble of nested dichotomies for multi-class scenarios is beneficial since it generates class likelihood estimations exclusively. To calculate the class probability for a specific leaf node in a multi-class problem, we can multiply the likelihood estimates generated by the two-class classifiers associated with each branch originating from the internal node of the base learner. One might argue for applying ordinary nested dichotomy

in this context. However, since each internal node's binary classifier is tasked with solving a unique two-class problem, multiple tree structures could be used for a given set of classes or generated from a dataset. Therefore, it is unclear which one to use without prior knowledge. It is not uncommon for two trees to provide contradictory results. That is, the predicted outcomes are affected by the tree structure used. For this reason, and since ensemble learning has proven effective in producing reliable predictions, it is rational to consider all potential nested dichotomies for a situation and then calculate the average of their likely estimations. Regrettably, achieving this is not feasible since the number of potential systems of nested dichotomies for an x -class issue is calculated as $(2x - 3)!!$ [58]. Thus, it is essential to choose a subset.

Some studies, such as [58], have proposed random sampling, subset selection and assigning equal probabilities to each tree as a possible solution. However, the effective performance of the random sampling method cannot be guaranteed, since any sampling strategy that favours one class label over another is unfair when no prior information is available. The use of uniformly distributed random samples across trees might be challenging for two reasons: 1) Constructing deep trees is time consuming, and 2) The unrestricted depth of the tree, which is only limited by the number of class labels.

Based on these findings, we proposed a data sampling enhanced EBND. The initial strategy involves ensuring that a comparable number of classes are present in all nodes. As an alternative to randomly selecting trees from the whole tree space, we constructed a balanced tree ensemble. The depth of the generated tree is guaranteed to be logarithmic for a particular number of class labels. An EBND is an ensemble of class-balanced nested dichotomies. In contrast to the overall number of nested dichotomies, there are fewer class-balanced alternatives. The total number of class-balanced trees is defined by the recurrence relation shown:

$$E(x) = \begin{cases} \frac{1}{2} \left(\frac{x}{x/2} \right) E\left(\frac{x}{2}\right) E\left(\frac{x}{2}\right) : x \text{ is even} \\ \left(\frac{x}{(x+1)/2} E\left(\frac{x+1}{2}\right) E\left(\frac{x-1}{2}\right) \right) : x \text{ is odd} \end{cases} \quad (5)$$

where $E(1) = E(2) = 1$.

Algorithm 4 presents the procedure for constructing a class balanced EBND. At each node, the class labels are divided into subgroups of about the same size, with slight variations if the number of classes is odd. The learning process is then applied to the data corresponding to these subsets. The algorithm repeats this procedure until only one class is left. To produce a committee (ensemble) of trees, it is applied several times with a variety of distinct random numbers as the seeds.

Furthermore, there may be cases where the dataset presents a high-class imbalance problem (CIP). That is, the class labels present in the dataset may be highly skewed and biased towards a specific label. A class-balanced tree does not

Algorithm 4 Ensemble Balanced Nested Dichotomy

Input:

- Training dataset T with instances (x, y) where x is a feature vector and y is the class label
- Set of classes C
- Number of dichotomies K
- Base classifier BC (FURIA, RIDOR)

Output:

- Ensemble of balanced nested dichotomies E
1. Initialize E as an empty list.
 2. For i from 1 to K :
 - a. Generate a balanced nested dichotomy ND_i for the set of classes C :
 - i. Initialize ND_i as an empty tree structure.
 - ii. Split classes C into two balanced subsets C_1 and C_2 :
 - Use a method to balance the class distribution between C_1 and C_2 (hierarchical clustering).
 - iii. Create a root node for ND_i with C_1 and C_2 as child nodes.
 - iv. Recursively split C_1 and C_2 until each subset contains a single class:
 - For each subset S :
 1. If S contains more than one class:
 - a. Split S into two balanced subsets S_1 and S_2 .
 - b. Create a node with S_1 and S_2 as child nodes.
 - c. Recursively repeat until each subset contains a single class.
 - b. Train a base classifier BC for each internal node in ND_i :
 - i. For each internal node n in ND_i with subsets S_1 and S_2 :
 - Create a binary training set T_n :
 1. Instances in T with class labels in S_1 are positive examples.
 2. Instances in T with class labels in S_2 are negative examples.
 - Train a base classifier BC on T_n and store it in n .
 - c. Add ND_i to E .
 3. Return E as the final ensemble of balanced nested dichotomies.

Procedure Classify (x, E) :

Input:

- Instance x to classify
- Ensemble of balanced nested dichotomies E

Output:

- Predicted class label for x

1. Initialize a votes dictionary to store class votes.
 2. For each dichotomy ND_i in E :
 - a. Start at the root node of ND_i .
 - b. While the current node is not a leaf node:
 - i. Use the base classifier BC stored in the current node to classify x .
 - ii. Move to the child node corresponding to the predicted subset.
 - c. The leaf node reached corresponds to the predicted class for ND_i .
 - d. Increment the vote count for the predicted class in the votes dictionary.
 3. Return the class with the highest vote count in the votes dictionary as the final predicted class label.
-

guarantee a data-balanced structure, where each of the two descendant nodes of an internal node has the same number of instances. If the basic learning approach creates a non-linear

structure based on the number of occurrences, it might negatively affect the tree's performance and accuracy. In this research, we introduced SMOTE to tackle latent CIP. Another reason for introducing the data balancing (SMOTE) approach is the need to ensure some degree of unpredictability in the allocation of class labels to subsets to keep the committee of randomly created EBND systems as diverse as possible.

D. SOFTWARE RISK PREDICTION DATASET

This study's experimental dataset is derived from software project risks. According to [43], it is made up of characteristics that are linked to both software risks and requirements. The dataset is generated by combining the SRS data with risk characteristics. Details on the project's risks, such as their categories, requirements, targets, probabilities, impacts, dimensions, and priorities, were gleaned from several sources [16], [29], [59]. As presented in Table 2, the dataset consists of 299 instances and 13 attributes split across five categories (class labels). Specifically, the dataset is derived from open-source projects that cut across multiple project categorisations, such as transaction processing systems, management information systems, enterprise systems, and safety critical systems. Instances based on the transaction processing system had the highest ratio of approximately 40% of the dataset, followed by those from management information systems with 29%, from enterprise systems with 20%, and safety critical systems with 11% of the instances. The stratification of the dataset shows its appropriateness for the research. In addition, the features of the dataset, such as the project category, requirement category, risk target, probability, magnitude of risk, impact, and dimension of risk are validated with findings from [16], [60], and [61]. The remaining features, such as risk fixing duration, cost, and project modules that may be affected, are validated by industrial experts (CMMI levels 2 and 3) with at least five years' experience. Additional information on the dataset may be found in [14], [43], [45], and [62].

E. EXPERIMENTAL APPROACH

This section outlines the approach used to conduct the experiments for this research. Figure 1 displays a schematic depiction of the experimental methodologies used in this investigation. The discussion of our approach is crucial as it provides empirical evidence supporting the success of the EBND fuzzy induction models that were devised and implemented. We conducted a two-stage experimental design in our research and objectively evaluated the prediction capabilities of the EBND fuzzy induction models that resulted from this approach.

Using the parameter settings outlined in Table 3, we compared FURIA and RIDOR against established rule-based models like Nearest Neighbour with Generalisation (NNGe), Partial Decision list (PART), and RIPPER, as well as high-performing ML models like Support Vector Machine (SVM), k-Nearest Neighbours (kNN), Decision Tree (DT), Aggregating One-Dependence Estimators (A1DE), Naive

Bayes (NB), and Random Forest (RF). These models were chosen for their demonstrated predictive abilities in software risk prediction studies and other ML applications. Furthermore, these models include a range of computational characteristics designed to enhance diversification and generalisation in empirical testing. The improved fuzzy induction techniques using the EBND idea were developed and used on the experimental dataset. EBND models were created utilising FURIA and RIDOR as basic learners. The proposed fuzzy induction models (EBNB-FURIA and EBND-RIDOR) used a committee of binary partition procedures to repeatedly separate each class label and create optimum subgroups for accurate classification. Moreover, the SMOTE approach is included in the prediction process to reduce the underlying class imbalance problem (CIP) and enhance the predictive performance of the EBND induction models (S-EBND-FURIA and S-EBND-RIDOR). The software risk prediction performances of the proposed fuzzy induction models (FURIA, RIDOR, EBND-FURIA, EBND-RIDOR, S-EBND-FURIA, and S-EBND-RIDOR) were assessed and contrasted with existing rule-based and ML-based models. An in-depth empirical study was required to analyse the prediction capabilities of the suggested methodologies outlined in Section I. This analysis included examining the empirical data collected from experiments and drawing conclusions from the observations. The software risk prediction models were developed by partitioning the dataset into training and testing sets using a 70-30 ratio. Cross-validation (CV) methods were used for model development. Stratified randomization was used to partition the datasets to minimize the chance of encountering duplicate patterns or values.

In this study, the CV utilizes a k value of 10 to construct the experimental models. The CV technique is preferred because past studies [63], [64], [65], [66], [67], [68] demonstrate the capacity of the CV strategy to withstand challenges of data quality that might result in model overfitting. The CV technique's ability to resist model overfitting is shown by its preference. Adhering to the principle of fairness, each experiment was repeated ten (10) times to maintain uniformity in the results produced by the models examined in this study. Ultimately, assessment of the models relies on the average values of the performance measure generated. The experiment used the WEKA [69] and the R packages [70] on a computer with sixteen GB of RAM and an Intel(R) Core CPU clocked at 3.4 GHz.

F. EVALUATION METRICS

To compare the predictive capacities of the different models that were tested, this study used several well-established assessment measures, such as accuracy, the f-measure, and Mathew's Correlation Coefficient (MCC). These metrics were chosen as the basis for selection criteria because they have been used extensively in previous research to evaluate existing models [35], [71], [72] and are considered credible since they include every part of the confusion matrix created by each developed model [73], [74].

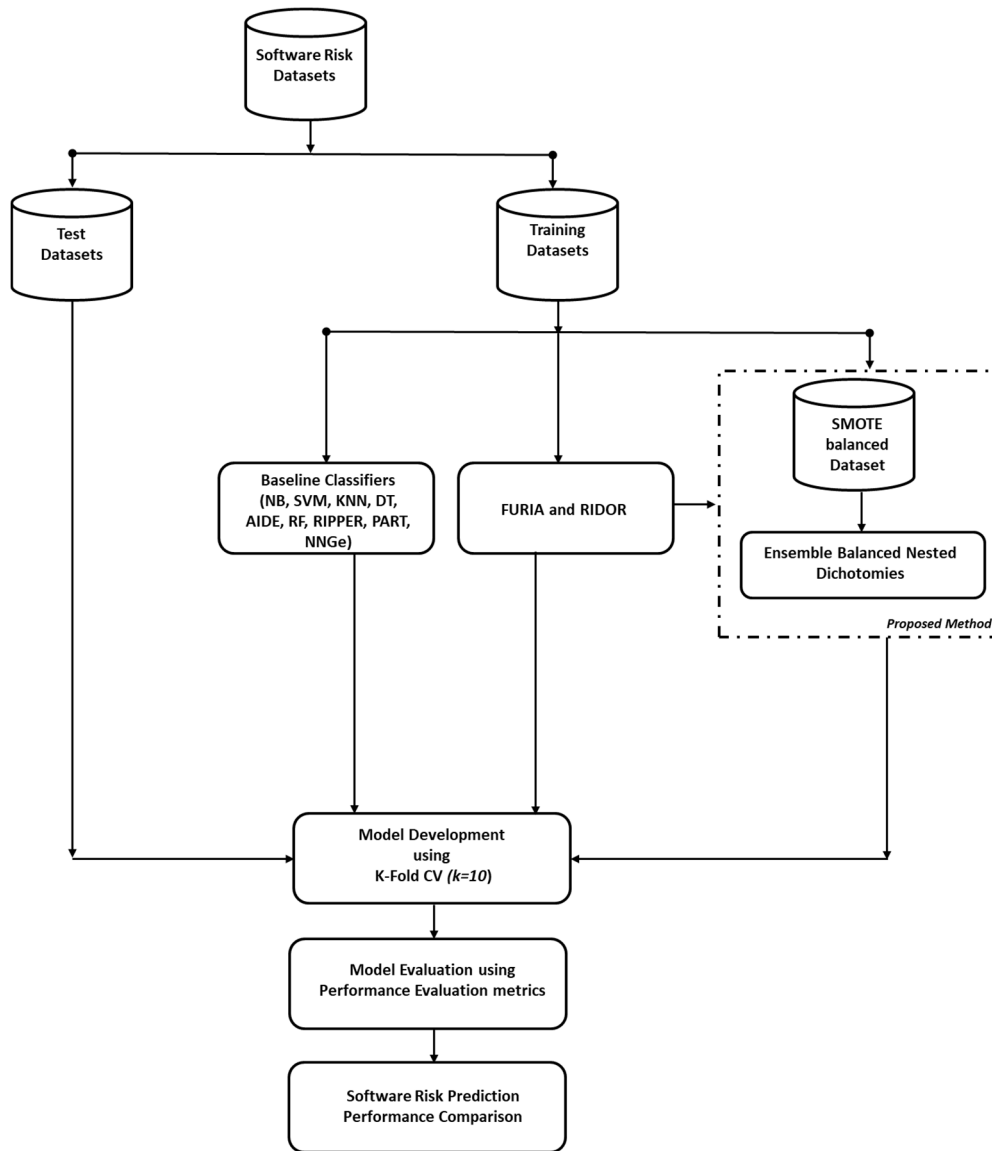


FIGURE 1. Experimental flow diagram.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section evaluates the effectiveness of the proposed fuzzy induction models, FURIA and RIDOR, in predicting software risks. We compare their performance with established models (listed in Table 2) on a common software risk prediction dataset. The aim is to assess whether FURIA and RIDOR outperform these benchmark models.

The experiments and findings are analysed and discussed in three distinct stages. This was done to enhance the clarity of the observed empirical results. The first scenario assesses the prediction capabilities of the developed fuzzy models (FURIA and RIDOR) in comparison to selected standard models (See Table 1). In the second scenario, the performance of FURIA and RIDOR together with their EBND variants (EBND-FURIA and EBND-RIDOR) were investigated and compared. The study also analyses the prediction accuracy of the improved data sampling-based EBND

models (S-EBNB-FURIA and S-EBND-RIDOR). Finally, in the third scenario, the prediction capabilities of the proposed and implemented models (FURIA, RIDOR, EBND-FURIA, EBND-RIDOR, S-EBNB-FURIA, and S-EBND-RIDOR) are evaluated against standard and current models used on the similar research data.

A. PHASE 1: COMPARATIVE ANALYSIS OF FURIA AND RIDOR VERSUS STANDARD MODELS

The predictive capabilities of FURIA and RIDOR were compared with distinct standard models (ML and rule-based classifiers) and are presented in Table 4. Our experiment demonstrated the effectiveness of the proposed fuzzy induction models, FURIA and RIDOR, in predicting software risks. Among the evaluated models, RIDOR emerged as the top performer, achieving an impressive accuracy of 98.33%. RIDOR equally outperformed the rest of the models in terms

TABLE 2. Description of the software requirement risk dataset.

Features	Datatype	Description
Requirements	String	Text
Project Category	Nominal	Transaction Processing System, Safety Critical System, Enterprise System, Management Information System
Requirement Category	Nominal	Functionality, Usability, Reliability & Availability, Performance, Security, Supportability, Constraints, Interfaces, Standards, Safety
Risk Target Category	Nominal	Budget, Quality, Schedule, Personal, Performance, Functional Validity, People, Project complexity, Planning & Control, Team, Resource availability, User, Requirement, Time Dimension, Organizational Environment, Cost, Design, Business, Unrealistic Requirements, Overdrawn Budget, Software, Process
Probability	Numeric	0–100%
Magnitude of Risk	Nominal	Negligible, Very Low, Low, Medium, High, Very High, Extreme
Impact	Nominal	high, catastrophic, moderate, Low, insignificant
Dimension of Risk	Numeric	Requirements, User, Project complexity, planning and control, Team, Organizational Environment, Estimations, Software Requirement, Planning and Control, Schedule, Complexity, Project cost, Organizational Requirements
Affecting No Modules	Numeric	Numbers (Whole Numbers)
Fixing Duration	Numeric	Days (Digits)
Fix cost	Numeric	Digits (0-9)
Priority	Numeric	0–100%
Risk Level	Nominal	1,2,3,4,5

TABLE 3. Parameter configuration for the implemented ML models.

Model	Model Type	Parameter Configuration
NNGe	Rule-based	numAttemptGeneOption=5; batchSize=100; numFolderMIOption=5
RIPPER		checkErrorRate=True; usePruning=True; NumFolds=3; optimizations=2
PART		NumFolds=3; useMDLCorrection=True; confidenceFactor=0.25; unpruned=False;
KNN	ML-based	DistanceFunction=EuclideanDistance; distanceWeighting=False; K=1; NearestNeighbourSearchAlgorithm = LinearNNSearch
NB		UseKernelEstimator=False; UseSupervisedDiscretization=False
SVM		shrinking=True; SVMType=C-SVC; loss=0.1; cost=0.1; nu=0.5; gamma=0.0; eps= 0.001; KernelType=RadialBasisFunction
DT		confidenceFactor=0.25; useMDLCorrection=True; subtreeRaising=True; collapseTree=True
A1DE		weight=1.0; frequencyLimit=1; subsumptionResolution=False; weightAODE=False;
RF	Fuzzy Rule-based	calcOutOfBag=False; bagSizePercent=100; numIterations=100
RIDOR		majorityClass=False; shuffle=1; wholeDataErr=False; minNo=2.0;
FURIA		minFolds=3; checkErrorRate=True; T-Norm=ProductT-Norm; optimization=2;

of f-measure and MCC with 0.983 and 0.979 respectively. While both FURIA and RIDOR performed well, RIDOR's superiority became evident when compared to established ML models like NB, A1DE, Support Vector SVM, KNN, DT, and RF. RIDOR consistently outperformed these models

in accuracy, with improvements ranging from +2.08% to a significant +18.07%.

Similarly, FURIA achieved higher accuracy values compared to the ML-based models we tested. RIDOR and FURIA beat the machine learning models in terms of assessment

TABLE 4. Comparison of FURIA and RIDOR with the standard ML models.

Model	Accuracy (%)	F-measure	MCC
RIDOR	98.33	0.983	0.979
FURIA	97.99	0.980	0.975
RIPPER	97.32	0.973	0.964
PART	96.65	0.967	0.956
DT	96.32	0.963	0.952
SVM	92.30	0.923	0.897
NB	90.97	0.911	0.877
AIDE	90.64	0.907	0.872
KNN	85.19	0.579	0.406
RF	83.28	0.805	0.766
NNGe	80.60	0.799	0.724

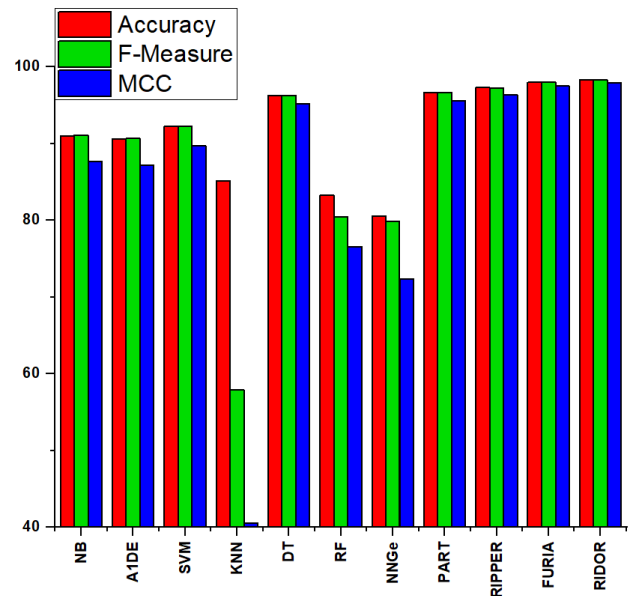
criteria like f-measure and MCC. The Decision Tree model fared the best among all machine learning models, whereas the K-Nearest Neighbours model performed the worst. KNN's low prediction performance is due to its process of lazy learning and reliance on parameter adjustment. The RIDOR and FURIA models fared better than the ML-based model in terms of the assessment measures used.

RIDOR and FURIA showed similar predictive results to NNGe, PART, and RIPPER in some instances, based on assessment measures. RIDOR had an increase of 1.74% and 1.04% in accuracy values compared to FURIA, which had an increase of 1.38% and 0.7% over PART and RIPPER, respectively. Identical trends were seen in the predicted accuracy of RIDOR and FURIA compared to the rule-based models based on f-measure and MCC values. RIPPER showed identical results to the proposed fuzzy models, although RIDOR and FURIA showed better prediction accuracy. RIPPER's outstanding prediction performance may be attributed to its divide-and-conquer operating strategy, which simplifies data instances categorization.

The utilization of fuzzy rules by RIDOR and FURIA surpasses traditional rules seen in RIPPER and RS. In addition, RIDOR and FURIA may use an unsorted rule set, with FURIA including an efficient rule stretching technique. Figure 2 shows the pictorial illustration of RIDOR and FURIA and implemented standard models. Detailed confusion matrices for FURIA and RIDOR are provided in Appendices 4 and 5 respectively. For further enhancement of the proposed fuzzy induction models, ensemble balanced nested dichotomy principles were incorporated into RIDOR and FURIA to enhance their prediction ability. The next section provides an in-depth analysis comparing the predictive abilities of RIDOR and FURIA together with their EBND variations (EBDN-FURIA, EBND-RIDOR, S-EBND-FURIA, and S-EBND-RIDOR) using the dataset under examination.

B. PHASE 2: PERFORMANCE ANALYSIS OF S-EBND-FURIA AND S-EBND-RIDOR AGAINST EBND-FURIA AND EBND-RIDOR AGAINST FURIA AND RIDOR

Table 5 displays the assessment values of RIDOR and FURIA and their respective EBND variants (EBND-FURIA,

**FIGURE 2.** Pictorial illustration of RIDOR and FURIA and implemented standard models.**TABLE 5.** FURIA and RIDOR versus its EBND variations.

Models	Accuracy (%)	F-measure	MCC
S-EBND-FURIA	99.41	0.994	0.993
S-EBND-RIDOR	99.41	0.991	0.989
RIDOR	98.33	0.983	.979
EBND-FURIA	98.33	0.983	0.979
EBND-RIDOR	98.33	0.983	0.979
FURIA	97.99	0.980	0.975

EBND-RIDOR, S-EBND-FURIA and S-EBND-RIDOR) on the studied dataset. In the first instance, as presented in Table 5, the suggested EBND variants of RIDOR and FURIA performed better than the regular FURIA and RIDOR models. Specifically, EBND-FURIA and EBND-RIDOR together achieved the best prediction accuracy of 99.41%, with 0.983 f-measure and 0.979 MCC, showing improvements of +0.34%, +0.31%, and +0.41% respectively compared to FURIA's predictive performance on the same dataset. It was also observed that there is no difference in the predictive performance of RIDOR and EBND-RIDOR. This finding can be attributed to the rule revision and refinement characteristics of RIDOR as it is capable of revising and refining its rules when presented with new instances that contradict the existing rules. This adaptability trait of RIDOR allows the model to improve its performance over time. Both EBND-FURIA and EBND-RIDOR achieved remarkably similar performances, with identical accuracy (98.33%), F-measure (0.983), and MCC (0.979) values. While the variations between these proposed fuzzy induction models and the original FURIA and RIDOR models might be negligible, it is crucial to remember that even a single misclassification in software risk prediction can have serious consequences.

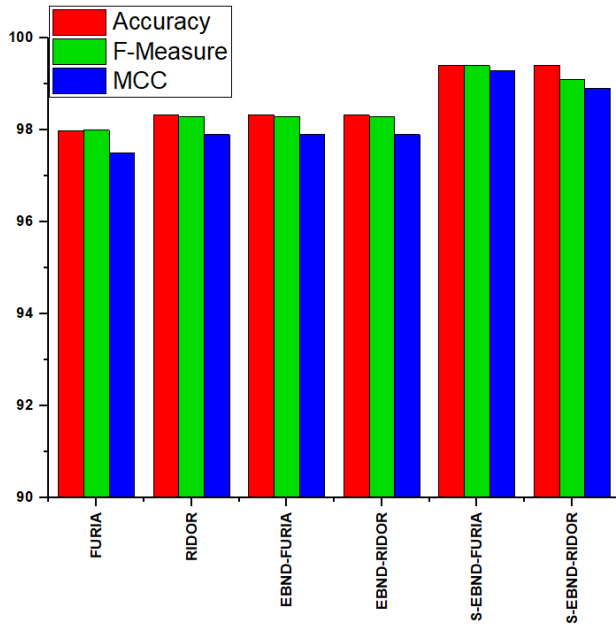


FIGURE 3. Pictorial illustration of RIDOR, FURIA and their EBND variations.

Furthermore, on the comparison of the predictive performances of S-EBND-RIDOR and S-EBND-FURIA, it can be observed that sampling-based EBND variants outperformed not only the baseline RIDOR and FURIA models but also EBND-FURIA and EBND-FURIA. Specifically, the duo of S-EBND-RIDOR and S-EBND-FURIA had a +1.45% and +1.09% increment in accuracy value over FURIA, RIDOR, EBND-FURIA and EBND-FURIA respectively. An identical increase is seen in the f-measure and MCC metric values as S-EBND-RIDOR and S-EBND-FURIA recorded positive improvements. This feat can be a result of handling the problem of class imbalance inherent in the dataset that might have affected the performances of FURIA and RIDOR. In addition, the enhanced predictive capabilities of the EBND variations (EBND-FURIA, EBND-FURIA, S-EBND-RIDOR and S-EBND-FURIA) can be ascribed to their capacity to effectively manage hierarchical class relationships in the experimental dataset. That is, the EBND variants were able to accommodate the overlapping class labels and data quality problems in the experiments that adversely affected the performance of implemented standard (rule and ML) models. Furthermore, the proposed EBND variants provide a kind of incremental learning, whereby the incorporation of new classes or instances into the current model does not require a total retraining of the whole model. Additionally, the use of incremental learning inside EBND operations has the potential to save computing resources and facilitate effective adaptation to dynamic situations. Figure 3 depicts the performances of FURIA and RIDOR with their respective EBND variations on the analysed software risk dataset.

C. PHASE 3: COMPARATIVE ASSESSMENT OF RIDOR, FURIA, AND THEIR EBND VARIATIONS VERSUS CURRENT MODELS

Comparative analysis of the proposed fuzzy induction models and current models in the literature, such as [14], [18], [13], [51], and [45] is shown in Table 6. An sample study by [14] used a hybrid approach that combined Decision Table and Naïve Bayes (DTNB) to estimate software risk. The results of this study demonstrated a competitive performance with an accuracy value of 97.99%. Even though DTNB exhibited comparable predictive performances when compared to the implemented FURIA and RIDOR models, the suggested S-EBND-RIDOR and S-EBND-FURIA versions showed superior predictive performance over DTNB. In the same study, [14] utilized a credal decision tree (CDT) for software risk prediction that had a comparable predictive performance as FURIA and RIDOR, yet the proposed S-EBND-RIDOR and S-EBND-FURIA outperformed the CDT model While the BNCC model developed by [18] achieved a 75.15% prediction accuracy, the proposed fuzzy induction models consistently outperformed it. Similarly, two other studies [22] and [45] used hyper-parameterized ANN, SVM, and NB classifiers, respectively. Reference [51] developed FEPP, an advanced decision forest technique. FEPP had a prediction performance of approximately 97% across the studied metrics. Nevertheless, the efficiency of these models is comparatively inferior to those of the RIDOR, FURIA, and their EBND variant models. Extensive confusion matrices for the enhanced fuzzy induction models are presented in Appendices 6, 7, 8, and 9 respectively.

TABLE 6. RIDOR, FURIA and their EBND variations versus current models.

Models	Accuracy (%)	F-measure	MCC
FURIA*	97.99	0.980	0.975
RIDOR*	98.33	0.983	0.979
EBND-FURIA*	98.33	0.983	0.979
EBND-RIDOR*	98.33	0.983	0.979
S-EBND-FURIA*	99.41	0.994	0.993
S-EBND-RIDOR*	99.41	0.991	0.989
[14] DTNB	97.99	0.980	0.959
[14] CDT	97.99	0.980	0.968
[45] NB	98.00	-	-
[18] BNCC	75.15	-	-
[22] ANN	75.00	-	-
[22] SVM	85.00	-	-
[51] FEPP	96.84	0.969	0.969

In summary, the EBND fuzzy induction models showed more precise predictions than the standard and existing ML and rule-based models based on diverse computing approaches on the same dataset. This can be attributed to the fact that EBND consists of a hierarchical structure composed of binary trees, where each node is linked to a certain group of classes. The classification procedure inside the internal nodes of the EBND tree utilizes a binary classifier to

partition the classes into two separate subsets. These subsets are then allocated to their corresponding child nodes. That is, the EBND system employs a hierarchical structure consisting of numerous nested dichotomies, with each tree being produced randomly. In addition, the use of ensemble principles to refine rules is another key factor that enhances the performance of EBND. Unlike traditional models, EBND prioritizes the extraction of accurate and concise rules by acknowledging that shorter rules are often easier to understand. This focus on rule length strengthens EBND's ability to comprehend the fundamental patterns in the experiments, while also enhancing the interpretability of the model.

In addition, EBND safeguards against the prevalence of rules from the majority class by considering each class independently throughout the rule extraction process, guaranteeing a fair assessment. To enhance its ability to make accurate predictions, EBND employs a robust class-balanced dichotomy method that involves systematically selecting features and optimising the generated binary tree structures. In addition, the capability of EBND to address different risk situations in software requirement is shown by its thorough study and incorporation of rules from different classes. One noteworthy aspect of the model is its proficiency in effectively managing unbalanced datasets. This ensures that the model's predictive capacity encompasses a wide range of risk levels. The integration method combines rules from many classes to create a comprehensive set of enhanced rules that capture the intricacies of software risk prediction. The impressive performance metric values achieved by EBND demonstrate its efficacy in striking a balance between minimising errors and accurately predicting positive scenarios. In this research, EBND models outperformed the benchmarked models due to its innovative algorithmic design, focus on succinct rules, and adaptability in different risk scenarios. For reproducibility, the experimental datasets, generated models, and solution buffers are available at the GitHub repository (<https://github.com/bharlow058/SoftwareRiskPrediction/tree/main>).

Practical use of the suggested fuzzy induction models involves the training of fuzzy models such as RIDOR or FURIA on gathered and pre-processed software requirement datasets. Thereafter, the implementation of a balanced nested dichotomy approach should be used, where the risk prediction problem is divided into binary decisions at each node of a hierarchical tree. For example, the first dichotomy could distinguish between high-risk and low-risk requirements. Subsequent dichotomies could further classify the risks into more specific categories, such as technical feasibility risk vs. requirement volatility risk. At each node in the nested dichotomy, fuzzy logic will be incorporated to handle uncertainty in the data. For instance, instead of a binary decision (risky or not risky), the model might assign a degree of riskiness (e.g., 0.7 risk level) based on fuzzy rules derived from the input features. The resulting dichotomies are then further aggregated, leading to a final risk score or category

for each requirement, which reflects the collective judgment of the ensemble dichotomies. The fuzzy logic component ensures that this risk assessment accounts for uncertainties and provides a more flexible, interpretable output. The development team can use these insights to re-evaluate the risky requirements, clarifying them with stakeholders and implementing measures to manage the identified risks.

V. THREAT TO VALIDITY

An essential aspect of experimental work is recognizing and addressing different threats and obstacles that might affect the overall applicability and reliability of the results acquired from the experiment [75], [76]. The current investigation has uncovered some possible challenges to the validity of the study, which are described in more detail below:

External validity: Ensuring the relevance of scientific research findings to practical situations is essential for assessing their dependability. The extent to which the findings of an experiment may be extrapolated to other settings may be constrained by several variables, including the implicit and explicit properties of the datasets used in the study. The current investigation makes use of a reference dataset that exemplifies a wide variety of characteristics in their entirety. The dataset, which is available to the public, is often used for the development and evaluation of software risk prediction models, and these models are easily and publicly accessible. The study extensively investigated the applied research method to ensure the replicability and accuracy of its methods on diverse software risks.

Internal validity: Internal validity may be defined as the extent to which an experimental approach captures the relevance and consistency of a body of facts, tried and proven models, and empirical study. Because of this, the study that is being conducted makes use of well-established rule-based and ML-based methods, which are techniques that have been successfully adopted and used in earlier research endeavours. These rule-based and ML-based techniques were chosen because they can include a wide variety of different computational approaches and because previous research has shown that they are effective at achieving good prediction performances. In addition, we used the CV approach to train the models we investigated on the designated dataset, and the process was closely monitored. To verify the reliability of each experiment, it was carried out ten times. The use of this strategy reduces the likelihood of discovering unexpected differences in the results of empirical research. On the other hand, more research might investigate other methods and procedures for evaluating models in upcoming research endeavours.

Construct validity: This notion refers to the evaluation requirements that were used to determine whether the software risk prediction models we investigated were effective. In the current study, many different statistical performance indicators were used. These metrics included accuracy, f-measure, and MCC, which made it possible to conduct an

in-depth empirical analysis of the models evaluated in the study. The models were meticulously designed with a focus on predicting software risks and their related features from the beginning.

VI. CONCLUSION

This study presents adaptive fuzzy induction techniques for effective and efficient prediction of risks or uncertainties in the SDLC. The suggested techniques involve the development and deployment of FURIA, RIDOR, and their EBND versions for predicting software requirements risks. FURIA and RIDOR are sophisticated fuzzy-based models that can generate highly predictive and robust models. They can handle both linear and non-linear data by incorporating pruning techniques for the generation of an optimal rule set that can be used for prediction purposes. Their respective EBND variants are based on the iterative deployment of a committee of binary split functions used to distinguish class labels and build optimum subsets for accurate prediction. The proposed models are assessed for practicality and efficiency using experiments carried out on a software risk dataset. The findings show that the proposed improved fuzzy models we developed are more effective than traditional standard models in predicting software risks across many classes.

Based on the findings, this study strongly recommends the use of fuzzy induction-based models for software risk prediction tasks. The fuzzy models have established prediction abilities and provide benefits like explainability, scalability, and stability, making them more favourable than standard models. An in-depth evaluation shows that the improved EBND models regularly exhibit greater predictive abilities when compared to standard models as presented in current literature using the same open-source dataset. This study suggests that using fuzzy induction-based models is a practical and efficient method for predicting software risks by highlighting their proven predictive abilities and the advantages of explainability, scalability, and stability over standard models. Also, the EBND variant models can efficiently operate on datasets consisting of uncertain values and inherent fuzziness.

The subsequent stage of this study will focus on perfecting nested dichotomies by studying alternate base learners together with numerous subset selection approaches. Additionally, it might be advantageous to choose subsets in a manner that maximizes ensemble variety, perhaps using information theoretic metrics such as variance of information. The use of established metaheuristic methodologies, such as genetic algorithms or other population-based metaheuristic methods may be extended to optimize ensembles via the construction of individual nested dichotomies. In addition, the effect of outliers and extreme values on the predictive prowess of standard models will be investigated. The capacity of a classifier to predict outcomes may be significantly impacted by either of these two concepts. Extreme values and outliers may distort the dataset, leading to an imbalanced and deeply

variable dataset, ultimately diminishing the predictability of the deployed model. Research has shown that classifiers respond unevenly to extreme values and outliers. It is therefore essential to analyse and understand the influence of outliers and extreme findings during data preprocessing and model selection.

TABLE OF SYMBOLS AND ABBREVIATIONS

Abbreviations/Symbols	Description
AIDE	Aggregating One-Dependence Estimators
ANN	Artificial Neural Network
BN	Bayesian Network
BNCC	Bayesian Network with Causality Constraints
BNN	Bayesian Neural Network
CDT	Cost sensitive Decision Tree
CHIRP	Continuous High-resolution Image Reconstruction using Patch priors
CIP	Class Imbalance Problem
CV	Cross Validation
DT*	Decision Tree
DTable	Decision Table
DTNB	Decision Tree and Naïve Bayes
EBND	Ensemble Balanced Nested Dichotomy
END	Ensemble Nested Dichotomy
FEPP	ForExPlusPlus
FURIA	Fuzzy Unordered Rule Induction Algorithm
GA	Genetic Algorithm
GNB	Gaussian Naïve Bayes
ID	Influence Diagrams
KNN	K Nearest Neighbour
LDA	Linear Discriminant Analysis
LR	Logistic Regression
ML	Machine Learning
MCC	Mathews Corellation Coefficient
MLR	Multiple Linear Regression
NB	Naïve Bayes
NNGe	Non-Nested Generalisation
PART	Partial Decision Tree
RF	Random Forest
RIDOR	Ripple Down Rule
RIPPER	Repeated Incremental Pruning to Produce Error Reduction
RPT	
SDLC	Software Development Life Cycle
SGD	Stochastic Gradient Descent
SMOTE	Synthetic Minority Oversampling Technique
SRS	Software Requirement Specification
SVM	Support Vector Machine
WEKA	Waikato Environment for Knowledge Analysis

APPENDIX

FURIA						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	28 9.36%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	28 100.00% 0.00%
2	0 0.00%	134 44.82%	1 0.33%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	74 24.75%	1 0.33%	0 0.00%	75 98.67% 1.33%
4	0 0.00%	0 0.00%	2 0.67%	42 14.05%	1 0.33%	45 93.33% 6.67%
5	0 0.00%	0 0.00%	0 0.00%	1 0.33%	15 5.02%	16 93.75% 6.25%
SUM	28 100.00% 0.00%	134 100.00% 0.00%	77 96.10% 3.90%	44 95.45% 4.55%	16 93.75% 6.25%	293 / 299 97.99% 2.01%

FIGURE 4. Pictorial illustration of confusion matrix for FURIA.

RIDOR						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	28 9.36%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	28 100.00% 0.00%
2	0 0.00%	134 44.82%	1 0.33%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	74 24.75%	1 0.33%	0 0.00%	75 98.67% 1.33%
4	0 0.00%	0 0.00%	1 0.33%	43 14.38%	1 0.33%	45 95.56% 4.44%
5	0 0.00%	0 0.00%	0 0.00%	1 0.33%	15 5.02%	16 93.75% 6.25%
SUM	28 100.00% 0.00%	134 100.00% 0.00%	76 97.37% 2.63%	45 95.56% 4.44%	16 93.75% 6.25%	294 / 299 98.33% 1.67%

FIGURE 5. Pictorial illustration of confusion matrix for RIDOR.

EBND-FURIA						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	28 9.36%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	28 100.00% 0.00%
2	0 0.00%	134 44.82%	1 0.33%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	74 24.75%	1 0.33%	0 0.00%	75 98.67% 1.33%
4	0 0.00%	0 0.00%	1 0.33%	43 14.38%	1 0.33%	45 95.56% 4.44%
5	0 0.00%	0 0.00%	0 0.00%	1 0.33%	15 5.02%	16 93.75% 6.25%
SUM	28 100.00% 0.00%	134 100.00% 0.00%	76 97.37% 2.63%	45 95.56% 4.44%	16 93.75% 6.25%	294 / 299 98.33% 1.67%

FIGURE 6. Pictorial illustration of confusion matrix for EBND-FURIA.

EBND-RIDOR						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	28 9.36%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	28 100.00% 0.00%
2	0 0.00%	134 44.82%	1 0.33%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	74 24.75%	1 0.33%	0 0.00%	75 98.67% 1.33%
4	0 0.00%	0 0.00%	1 0.33%	43 14.38%	1 0.33%	45 95.56% 4.44%
5	0 0.00%	0 0.00%	0 0.00%	1 0.33%	15 5.02%	16 93.75% 6.25%
SUM	28 100.00% 0.00%	134 100.00% 0.00%	76 97.37% 2.63%	45 95.56% 4.44%	16 93.75% 6.25%	294 / 299 98.33% 1.67%

FIGURE 7. Pictorial illustration of confusion matrix for EBND-RIDOR.

S-EBND-FURIA						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	135 20.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	135 100.00% 0.00%
2	0 0.00%	134 19.85%	1 0.15%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	134 19.85%	1 0.15%	0 0.00%	135 99.26% 0.74%
4	0 0.00%	0 0.00%	0 0.00%	133 19.70%	2 0.30%	135 98.52% 1.48%
5	0 0.00%	0 0.00%	0 0.00%	0 0.00%	135 20.00%	135 100.00% 0.00%
SUM	135 100.00% 0.00%	134 100.00% 0.00%	135 99.26% 0.74%	134 99.25% 0.75%	137 98.54% 1.46%	671 / 675 99.41% 0.59%

FIGURE 8. Pictorial illustration of confusion matrix for S-EBND-FURIA.

S-EBND-RIDOR						
TARGET \ OUTPUT	1	2	3	4	5	SUM
1	135 20.00%	0 0.00%	0 0.00%	0 0.00%	0 0.00%	135 100.00% 0.00%
2	0 0.00%	134 19.85%	1 0.15%	0 0.00%	0 0.00%	135 99.26% 0.74%
3	0 0.00%	0 0.00%	134 19.85%	1 0.15%	0 0.00%	135 99.26% 0.74%
4	0 0.00%	0 0.00%	0 0.00%	133 19.70%	2 0.30%	135 98.52% 1.48%
5	0 0.00%	0 0.00%	0 0.00%	0 0.00%	135 20.00%	135 100.00% 0.00%
SUM	135 100.00% 0.00%	134 100.00% 0.00%	135 99.26% 0.74%	134 99.25% 0.75%	137 98.54% 1.46%	671 / 675 99.41% 0.59%

FIGURE 9. Pictorial illustration of confusion matrix for S-EBND-RIDOR.

ACKNOWLEDGMENT

The authors wish to express their appreciation to the School of Digital Sciences, Universiti Brunei Darussalam, and the reviewers for their helpful suggestions which greatly improved the presentation and quality of this article.

REFERENCES

- [1] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekk, and D. Doermann, "Future of software development with generative AI," *Automated Softw. Eng.*, vol. 31, no. 1, p. 26, May 2024.
- [2] R. Bocu, A. Baicoianu, and A. Kerestely, "An extended survey concerning the significance of artificial intelligence and machine learning techniques for bug triage and management," *IEEE Access*, vol. 11, pp. 123924–123937, 2023.
- [3] H. W. Marar, "Advancements in software engineering using AI," *Comput. Softw. Media Appl.*, vol. 6, no. 1, p. 3906, Feb. 2024.
- [4] N. Ilame, "Machine learning-powered programming: Exploring the fusion of AI and coding," *Innov. Comput. Sci. J.*, vol. 10, no. 1, pp. 1–11, 2024.
- [5] R. Wang, R. Cheng, D. Ford, and T. Zimmermann, "Investigating and designing for trust in AI-powered code generation tools," in *Proc. ACM Conf. Fairness, Accountability, Transparency*, Jun. 2024, pp. 1475–1493.
- [6] R. Yıldırım, K. Aydın, and O. Çetin, "Evaluating the impact of conventional code analysis against large language models in API vulnerability detection," in *Proc. Eur. Interdiscipl. Cybersecurity Conf.*, Jun. 2024, pp. 57–64.
- [7] I. Ozkaya, "The golden age of software engineering [from the editor]," *IEEE Softw.*, vol. 36, no. 1, pp. 4–10, Jan. 2019.
- [8] M. Krichen, "How artificial intelligence can revolutionize software testing techniques," in *Proc. Int. Conf. Innov. Bio-Inspired Comput. Appl.* Cham, Switzerland: Springer, 2022, pp. 189–198.
- [9] C. Augusto, J. Morán, C. D. L. Riva, and J. Tuya, "Test-driven anonymization for artificial intelligence," in *Proc. IEEE Int. Conf. Artif. Intell. Test. (AITest)*, Apr. 2019, pp. 103–110.
- [10] I. M. Alharbi, A. A. Alyoubi, M. Altuwairiqi, and M. A. Ellatif, "Analysis of risks assessment in multi software projects development environment using classification techniques," presented at the Int. Conf. Adv. Mach. Learn. Technol. Appl., Cairo, Egypt, 2021.
- [11] H. A. M. Mohamed, "Model-based prediction of resource utilization and performance risks," Ph.D. dissertation, Sudan Univ. Sci. Technol., Sudan, Sudan, 2018.
- [12] H. A. M. Salih and H. H. Ammar, "Model-based resource utilization and performance risk prediction using machine learning techniques," *Int. J. Informat. Vis.*, vol. 1, no. 3, pp. 101–109, Jul. 2017.
- [13] Y. Hu, B. Feng, X. Mo, X. Zhang, E. W. T. Ngai, M. Fan, and M. Liu, "Cost-sensitive and ensemble-based prediction model for outsourced software project risk prediction," *Decis. Support Syst.*, vol. 72, pp. 11–23, Apr. 2015.
- [14] R. Naseem, Z. Shaukat, M. Irfan, M. A. Shah, A. Ahmad, F. Muhammad, A. Glowacz, L. Dunai, J. Antonino-Daviu, and A. Sulaiman, "Empirical assessment of machine learning techniques for software requirements risk prediction," *Electronics*, vol. 10, no. 2, p. 168, Jan. 2021.
- [15] A. Aslam, N. Ahmad, T. Saba, A. S. Almazayad, A. Rehman, A. Anjum, and A. Khan, "Decision support system for risk assessment and management strategies in distributed software development," *IEEE Access*, vol. 5, pp. 20349–20373, 2017.
- [16] L. Williams, "Project risks product-specific risks," *J. Secur. NCSU*, vol. 1, no. 1, pp. 1–22, 2004.
- [17] R. T. Mahmood and I. A. Saleh, "Software risk prediction through the use of machine learning," *Int. Res. J. Innov. Eng. Technol.*, vol. 7, no. 2, p. 42, 2023.
- [18] Y. Hu, X. Zhang, E. W. T. Ngai, R. Cai, and M. Liu, "Software project risk analysis using Bayesian networks with causality constraints," *Decis. Support Syst.*, vol. 56, pp. 439–449, Dec. 2013.
- [19] S. Du, M. Keil, L. Mathiassen, Y. Shen, and A. Tiwana, "Attention-shaping tools, expertise, and perceived control in IT project risk assessment," *Decis. Support Syst.*, vol. 43, no. 1, pp. 269–283, Feb. 2007.
- [20] C.-F. Fan and Y.-C. Yu, "BBN-based software project risk management," *J. Syst. Softw.*, vol. 73, no. 2, pp. 193–203, Oct. 2004.
- [21] D. E. Neumann, "An enhanced neural network technique for software risk analysis," *IEEE Trans. Softw. Eng.*, vol. 28, no. 9, pp. 904–912, Sep. 2002.
- [22] Y. Hu, X. Zhang, X. Sun, M. Liu, and J. Du, "An intelligent model for software project risk prediction," presented at the Int. Conf. Inf. Manag., Innov. Manag. Ind. Eng., Xi'an, China, 2009.
- [23] C.-G. Bai, "Bayesian network based software reliability prediction with an operational profile," *J. Syst. Softw.*, vol. 77, no. 2, pp. 103–112, Aug. 2005.
- [24] E. Lee, Y. Park, and J. G. Shin, "Large engineering project risk management using a Bayesian belief network," *Expert Syst. Appl.*, vol. 36, no. 3, pp. 5880–5887, Apr. 2009.
- [25] T. M. Khoshgoftaar, E. B. Allen, J. P. Hudepohl, and S. J. Aud, "Application of neural networks to software quality modeling of a very large telecommunications system," *IEEE Trans. Neural Netw.*, vol. 8, no. 4, pp. 902–909, Jul. 1997.
- [26] A. O. Balogun, S. Basri, S. J. Abdulkadir, and A. S. Hashim, "Performance analysis of feature selection methods in software defect prediction: A search method approach," *Appl. Sci.*, vol. 9, no. 13, p. 2764, Jul. 2019.
- [27] A. O. Balogun, S. Basri, S. Mahamad, S. J. Abdulkadir, M. A. Almomani, V. E. Adeyemo, Q. Al-Tashi, H. A. Mojeed, A. A. Imam, and A. O. Bajeh, "Impact of feature selection methods on the predictive performance of software defect prediction models: An extensive empirical study," *Symmetry*, vol. 12, no. 7, p. 1147, Jul. 2020.
- [28] A. O. Balogun, S. Basri, J. A. Said, V. E. Adeyemo, A. A. Imam, and A. O. Bajeh, "Software defect prediction: Analysis of class imbalance and performance stability," *J. Eng. Sci. Technol.*, vol. 14, no. 6, pp. 3294–3308, 2019.
- [29] J. Hühn and E. Hüllermeier, "FURIA: An algorithm for unordered fuzzy rule induction," *Data Mining Knowl. Discovery*, vol. 19, no. 3, pp. 293–319, Dec. 2009.
- [30] S. Mejjaoui and S. Guizani, "PDF malware detection based on fuzzy unordered rule induction algorithm (FURIA)," *Appl. Sci.*, vol. 13, no. 6, p. 3980, Mar. 2023.
- [31] S. Kalmegh and S. Deshmukh, "Categorical identification of Indian news using J48 and Ridor algorithm," *Int. Refereed J. Eng. Sci.*, vol. 3, no. 6, pp. 79–84, 2014.
- [32] A. M. Aubaid and A. Mishra, "A rule-based approach to embedding techniques for text document classification," *Appl. Sci.*, vol. 10, no. 11, p. 4009, Jun. 2020.
- [33] M. Wever, F. Mohr, and E. Hüllermeier, "Ensembles of evolved nested dichotomies for classification," presented at the Genetic Evol. Comput. Conf., Kyoto, Japan, Jul. 2018.
- [34] A. O. Balogun, F. B. Lafenwa-Balogun, H. A. Mojeed, V. E. Adeyemo, O. N. Akande, A. G. Akintola, A. O. Bajeh, and F. E. Usman-Hamza, "SMOTE-based homogeneous ensemble methods for software defect prediction," presented at the Int. Conf. Comput. Sci. Appl., Cagliari, Italy, 2020.
- [35] V. E. Adeyemo, A. O. Balogun, H. A. Mojeed, N. O. Akande, and K. S. Adewole, "Ensemble-based logistic model trees for website phishing detection," presented at the Int. Conf. Adv. Cyber Secur., Penang, Malaysia, 2020.
- [36] A. O. Balogun, F. B. Lafenwa-Balogun, H. A. Mojeed, F. E. Usman-Hamza, A. O. Bajeh, V. E. Adeyemo, K. S. Adewole, and R. G. Jimoh, "Data sampling-based feature selection framework for software defect prediction," presented at the Int. Conf. Emerg. Appl. Technol. Ind. 4.0, Akwa Ibom, Nigeria, 2021.
- [37] T. Khan and M. Faisal, "An efficient Bayesian network model (BNM) for software risk prediction in design phase development," *Int. J. Inf. Technol.*, vol. 15, no. 4, pp. 2147–2160, Apr. 2023.
- [38] T. Kawamura, T. Toma, and K. Takano, "Outcome prediction of software projects for information technology vendors," presented at the IEEE Int. Conf. Ind. Eng. Eng. Manage. (IIEEM), Singapore, Dec. 2017.
- [39] T. Christiansen, P. Wuttidittachotti, S. Prakancharoen, and S. A.-O. Vallipakorn, "Prediction of risk factors of software development project by using multiple logistic regression," *ARPN J. Eng. Appl. Sci.*, vol. 10, no. 3, pp. 1324–1331, 2015.
- [40] B. Khan, R. Naseem, I. Alam, I. Khan, H. Alasmay, and T. Rahman, "Analysis of tree-family machine learning techniques for risk prediction in software requirements," *IEEE Access*, vol. 10, pp. 98220–98231, 2022.
- [41] M. S. G. Qureshi, B. Khan, and M. Arshad, "ML-based model for risk prediction in software requirements," *Int. J. Technol. Diffusion*, vol. 13, no. 1, pp. 1–17, Nov. 2022.
- [42] M. H. Mahmud, M. T. H. Nayan, D. M. N. A. Ashir, and M. A. Kabir, "Software risk prediction: Systematic literature review on machine learning techniques," *Appl. Sci.*, vol. 12, no. 22, p. 11694, Nov. 2022.

- [43] Z. S. Shaikat, R. Naseem, and M. Zubair, "A dataset for software requirements risk prediction," presented at the IEEE Int. Conf. Comput. Sci. Eng. (CSE), Bucharest, Romania, Oct. 2018.
- [44] Z. Xu, B. Yang, and P. Guo, "Software risk prediction based on the hybrid algorithm of genetic algorithm and decision tree," presented at the Int. Conf. Intell. Comput., Qingdao, China, 2007.
- [45] B. O. Akumba, S. U. Otor, I. Agaji, and B. T. Akumba, "A predictive risk model for software projects' requirement gathering phase," *Int. J. Innov. Sci. Res. Technol.*, vol. 5, no. 6, pp. 231–236, Jun. 2020.
- [46] M. M. Ootom, "ABMJ: An ensemble model for risk prediction in software requirements," *Int. J. Comput. Sci. Netw. Secur.*, vol. 22, no. 3, p. 710, 2022.
- [47] H. Mamman, A. O. Balogun, S. Basri, L. F. Capretz, V. E. Adeyemo, A. A. Imam, and G. Kumar, "Software requirement risk prediction using enhanced fuzzy induction models," *Electronics*, vol. 12, no. 18, p. 3805, Sep. 2023.
- [48] C. Maszczyk, M. Sikora, and Ł. Wróbel, "Classification, regression, and survival rule induction with complex and M-of-N elementary conditions," *Mach. Learn. Knowl. Extraction*, vol. 6, no. 1, pp. 554–579, Mar. 2024.
- [49] S. M. Avdoshin and E. Y. Pesotskaya, "Software risk management," in *Proc. Eur. Softw. Eng. Conf.* Cham, Switzerland: Springer, Oct. 2011, pp. 1–6.
- [50] Y. P. Vera and Á. F. Del Carpio, "Comparing machine learning techniques for software requirements risk prediction," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 34, no. 1, p. 508, Apr. 2024.
- [51] M. Binsawad and B. Khan, "FEPP: Advancing software risk prediction in requirements engineering through innovative rule extraction and multi-class integration," *IEEE Access*, vol. 12, pp. 59851–59860, 2024.
- [52] Y. Li, I. J. Pérez, F. J. Cabrerizo, H. Garg, and J. A. Morente-Molinera, "A belief rule-based classification system using fuzzy unordered rule induction algorithm," *Inf. Sci.*, vol. 667, May 2024, Art. no. 120462.
- [53] T. Zhang, Q. Fu, C. Li, F. Liu, H. Wang, L. Han, R. P. Quevedo, T. Chen, and N. Lei, "Modeling landslide susceptibility using data mining techniques of kernel logistic regression, fuzzy unordered rule induction algorithm, SysFor and random forest," *Natural Hazards*, vol. 114, no. 3, pp. 3327–3358, Dec. 2022.
- [54] B. R. Gaines and P. Compton, "Induction of ripple-down rules applied to modeling large databases," *J. Intell. Inf. Syst.*, vol. 5, no. 3, pp. 211–228, Nov. 1995.
- [55] W. N. H. W. Mohamed, M. N. M. Salleh, and A. H. Omar, "A comparative study of reduced error pruning method in decision tree algorithms," presented at the IEEE Int. Conf. Control Syst., Comput. Eng., Penang, Malaysia, Nov. 2012.
- [56] WEKA Library. (WEKA). *Ripple-Down Rules (RIDOR)*. Accessed Aug. 7, 2023. [Online]. Available: <https://weka.sourceforge.io/doc/packages/ridor/weka/classifiers/rules/Ridor.html>
- [57] P. Compton and B. H. Kang, *Ripple-Down Rules: The Alternative to Machine Learning*. Boca Raton, FL, USA: CRC Press, 2021.
- [58] E. Frank and S. Kramer, "Ensembles of nested dichotomies for multi-class problems," presented at the 21st Int. Conf. Mach. Learn., Banff, AB, Canada, 2004.
- [59] B. W. Boehm, "Software risk management: Principles and practices," *IEEE Softw.*, vol. 8, no. 1, pp. 32–41, Jan. 1991.
- [60] T. Moores and R. Champion, "A methodology for measuring the risk associated with a software requirements specification," *Australas. J. Inf. Syst.*, vol. 4, no. 1, Nov. 1996.
- [61] B. Boehm, "Software risk management," presented at the Eur. Softw. Eng. Conf., Coventry, U.K., 1989.
- [62] S. Zain, N. Rashid, and Z. Muhammad. *Software Requirement Risk Prediction Dataset*. Accessed: Feb. 2, 2023. [Online]. Available: <https://zenodo.org/record/1209601>
- [63] A. O. Balogun, K. S. Adewole, A. O. Bajeh, and R. G. Jimoh, "Cascade generalization based functional tree for website phishing detection," presented at the Int. Conf. Adv. Cyber Secur., Penang, Malaysia, 2021.
- [64] A. O. Balogun, K. S. Adewole, M. O. Raheem, O. N. Akande, F. E. Usman-Hamza, M. A. Mabayoje, A. G. Akintola, A. W. Asaju-Gbolagade, M. K. Jimoh, R. G. Jimoh, and V. E. Adeyemo, "Improving the phishing website detection using empirical analysis of function tree and its variants," *Heliyon*, vol. 7, no. 7, Jul. 2021, Art. no. e07437.
- [65] A. O. Balogun, A. O. Bajeh, V. A. Orie, and A. W. Yusuf-Asaju, "Software defect prediction using ensemble learning: An ANP based evaluation method," *FUOYE J. Eng. Technol.*, vol. 3, no. 2, pp. 50–55, Sep. 2018.
- [66] A. O. Balogun, S. Basri, S. A. Jadid, S. Mahamad, M. A. Al-momani, A. O. Bajeh, and A. K. Alazzawi, "Search-based wrapper feature selection methods in software defect prediction: An empirical analysis," presented at the Comput. Sci. Conf., Zlin, Czech Republic, 2020.
- [67] A. O. Balogun, H. A. Mojeed, K. S. Adewole, A. G. Akintola, S. A. Salihu, A. O. Bajeh, and R. G. Jimoh, "Optimized decision forest for website phishing detection," presented at the Comput. Methods Syst. Softw., 2021.
- [68] L. Zhang, J. Wang, W. Wang, Z. Jin, C. Zhao, Z. Cai, and H. Chen, "A novel smart contract vulnerability detection method based on information graph and ensemble learning," *Sensors*, vol. 22, no. 9, p. 3581, May 2022.
- [69] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The WEKA data mining software: An update," *ACM SIGKDD Explor. Newslett.*, vol. 11, no. 1, pp. 10–18, Nov. 2009, doi: 10.1145/1656274.1656278.
- [70] M. J. Crawley, *The R Book*. Hoboken, NJ, USA: Wiley, 2012.
- [71] R. Jimoh, A. Balogun, A. Bajeh, and S. Ajayi, "A PROMETHEE based evaluation of software defect predictors," *J. Comput. Sci. Appl.*, vol. 25, no. 1, pp. 106–119, 2018.
- [72] B. J. Odejide, A. O. Bajeh, A. O. Balogun, Z. O. Alanamu, K. S. Adewole, A. G. Akintola, S. A. Salihu, F. E. Usman-Hamza, and H. A. Mojeed, "An empirical study on data sampling methods in addressing class imbalance problem in software defect prediction," presented at the Comput. Sci. Conf., Zlin, Czech Republic, 2022.
- [73] A. G. Akintola, A. Balogun, H. A. Mojeed, F. Usman-Hamza, S. A. Salihu, K. S. Adewole, G. B. Balogun, and P. O. Sadiku, "Performance analysis of machine learning methods with class imbalance problem in Android malware detection," *Int. J. Interact. Mobile Technol.*, vol. 16, no. 10, pp. 140–162, May 2022.
- [74] A. O. Balogun, B. J. Odejide, A. O. Bajeh, Z. O. Alanamu, F. E. Usman-Hamza, H. O. Adeleke, M. A. Mabayoje, and S. R. Yusuff, "Empirical analysis of data sampling-based ensemble methods in software defect prediction," presented at the Int. Conf. Comput. Sci. Appl., Malaga, Spain, 2022.
- [75] A. G. Akintola, A. O. Balogun, L. F. Capretz, H. A. Mojeed, S. Basri, S. A. Salihu, F. E. Usman-Hamza, P. O. Sadiku, G. B. Balogun, and Z. O. Alanamu, "Empirical analysis of forest penalizing attribute and its enhanced variations for Android malware detection," *Appl. Sci.*, vol. 12, no. 9, p. 4664, May 2022.
- [76] F. E. Usman-Hamza, A. O. Balogun, L. F. Capretz, H. A. Mojeed, S. Mahamad, S. A. Salihu, A. G. Akintola, S. Basri, R. T. Amosa, and N. K. Salahdeen, "Intelligent decision forest models for customer churn prediction," *Appl. Sci.*, vol. 12, no. 16, p. 8270, Aug. 2022.



GANESH KUMAR received the Ph.D. degree in information technology from Universiti Teknologi PETRONAS under the funding of Petronas Sdn. Bhd. and YUTP. He is currently a Lecturer with the Department of Computing and Information Systems, School of Engineering and Technology, Sunway University, Malaysia. Previously, he was a Graduate Assistant with Universiti Teknologi PETRONAS and a Lecturer with prestigious institutions in Karachi, Pakistan. He has overall more than ten years of teaching experience. He has more than 20 articles in international and national publication venues. He has academic experience of more than ten years along with two years of industrial experience. His research interests include natural language processing, software engineering, machine learning, and data analytics. He is also a Reviewer of IEEE Access and BMC.



ABDULLAHI ABUBAKAR IMAM is currently a Faculty Member and a Research and PG Coordinator with the School of Digital Science, Universiti Brunei Darussalam. He earlier worked with the Petronas University of Technology, Malaysia, Ahmadu Bello University, Zaria, Islamic University, Uganda, and United Nations AFRI, Uganda. He is a Computer Programmer, who has developed more than 20 software applications using different programming languages. He has published many journal articles, conferences, and book chapters. His areas of research interests include software engineering, artificial intelligence (AI), machine learning, data science, databases, and info security. He was a recipient of many local and international research grants and awards. He serves as a guest editor and a reviewer for many reputable journals.



SHUIB BASRI (Member, IEEE) received the bachelor's degree (Hons.) in IT (BIT) from Universiti Utara Malaysia, the master's degree in software system engineering (MSSE) from The University of Melbourne, Australia, and the Ph.D. degree in software engineering from Dublin City University, Republic of Ireland. He joined academia, after being an IT Engineer of basic and application systems with several IT-based companies for several years. Currently, he is a Senior Lecturer with the Petronas University of Technology and the Leader of the Software Quality and Quality Engineering Research Group, UTP. His research interests include software quality, software process improvement, agile methods, big data analytics, knowledge engineering, management engineering, and several more areas in software engineering. He is a member of ISO WG24 and the IEEE Computer Society.



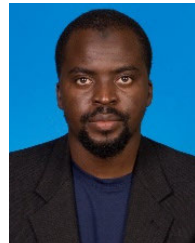
AHMAD SOBRI HASHIM received the Bachelor of Technology (B.Tech.), Master of Science (M.Sc.), and Doctor of Philosophy (Ph.D.) degrees in information technology from Universiti Teknologi PETRONAS (UTP). Since 2014, he has been teaching and he is very passionate in teaching programming courses, including structured programming and web programming. He is currently a Senior Lecturer with the Computer and Information Sciences Department (CISD), UTP. He has published quite a number of research articles, since 2009, and most of the articles are indexed by ISI and SCOPUS. His research interests include software engineering, IT applications, e-systems, human-computer interaction, educational technologies, learning disabilities, and information systems. He won the Gold Medal from Malaysia Technology Expo (MTE), in 2017, the International Competition and Exhibition on Computing Innovation (ICE-CInno), in 2016, the Pertandingan Rekacipta dan Inovasi Institusi Pengajian Tinggi Swasta (PERISTIS), in 2016, and the International Invention, Design and Articulation (i-IDEA), in 2016.



ABDUL GHANI HAJI NAIM received the Ph.D. degree in information security from the Royal Holloway College, University of London. In 2017, he was the Director of the Institute of Applied Data Analytics, Universiti Brunei Darussalam (UBD). He is currently a Senior Assistant Professor, who has been working with UBD, for more than 20 years as a Teaching Staff in computer science. He is with the Institute of Applied Data Analytics, UBD, researchers spearheading research on data analytics for various fields, such as bioinformatics and biodiversity, data visualization, the IoT, and cyber security. He is the Director of the Information Communication Technology Centre (ICTC). He has taught a number of computer science courses, such as computer architecture, operating systems, computer security, multimedia development, multimedia networks, and computer graphics. His area of specialization is in cryptography, especially on block ciphers. His research interests include cybersecurity, cryptography, and high-performance computing.



LUIZ FERNANDO CAPRETZ (Senior Member, IEEE) has vast experience in the software engineering field as a Practitioner, a Manager, and an Educator. Before joining Western University, Canada, he worked at both technical and managerial levels, taught, and did research on the engineering of software in Brazil, Argentina, U.K., Japan, United Arab Emirates, Malaysia, and Singapore. He is currently a Full Professor of software engineering and the Director of the Software Engineering Program. He has held visiting positions with the University of Sharjah, New York University, Abu Dhabi, Universiti Teknologi PETRONAS, and Yale-NUS College. His research interests include software engineering, human aspects of software engineering, software testing, and generative AI for software engineering. For further information please visit: http://eng.uwo.ca/electrical/faculty/capretz_l/



ABDULLATEEF OLUWAGBEMIGA BALOGUN (Member, IEEE) received the Ph.D. degree in information technology from Universiti Teknologi PETRONAS, Malaysia, in 2022. His Ph.D. thesis focused on the high dimensionality problem in software defect prediction. He is currently a Lecturer with Universiti Teknologi PETRONAS. Previously, he was with the University of Ilorin, Ilorin, Nigeria. He has more than 70 publications in peer-reviewed reputable journals and conferences. His research interests include empirical software engineering, machine learning, data analytics, blockchain, and cloud computing.



HUSSAINI MAMMAN received the B.Tech. degree in information technology and the M.Sc. degree in management information technology from Abubakar Tafawa Balewa University (ATBU), Nigeria, in 2010 and 2018, respectively. He is currently pursuing the Ph.D. degree with Universiti Teknologi PETRONAS, Malaysia. He is passionate about machine learning and software engineering. His academic journey began with ATBU. His research focuses on developing robust fairness testing frameworks for machine learning software. He is dedicated to promoting ethical and unbiased AI solutions. His commitment to this field is evident through his published scholarly works in journals and conferences. He aims to leverage his expertise in both software engineering and machine learning to create innovative solutions that ensure AI systems are transparent, inclusive, and free from harmful biases.

...